

Reflected XSS into Attribute with Angle Brackets HTML-Encoded

Lab Overview

This lab demonstrates a **Reflected Cross-Site Scripting (XSS)** vulnerability where user-controlled input is reflected into an HTML attribute.

Although angle brackets are HTML-encoded by the application, the input remains exploitable because the context is an HTML attribute.

Objective

Identify the reflected input and exploit the attribute context to execute JavaScript.

Vulnerability

Vulnerability Type: Reflected Cross-Site Scripting (XSS)

The application reflects attacker-controlled input into an HTML attribute without sufficiently preventing attribute-context injection.

Methodology

1. Open the lab application.
2. Locate a functionality that accepts user-controlled input.
3. Submit a test string and observe where the value appears in the response.
4. Inspect the resulting HTML.
5. Determine that the input is reflected inside an HTML attribute.
6. Observe that `<` and `>` are HTML-encoded.
7. Test whether the attribute can be escaped using a double quote.
8. Inject an event handler into the resulting HTML attribute.
9. Confirm JavaScript execution.

Exploitation

Because the input is placed inside an HTML attribute, the attack can escape the existing attribute using a quote.

A representative payload is:

```
" autofocus onfocus=alert(document.domain) x="
```

The payload attempts to:

1. Close the existing attribute.
2. Add an event handler.
3. Trigger JavaScript execution when the element receives focus.

Result

The injected event handler executes JavaScript in the victim's browser, demonstrating successful reflected XSS.

Impact

A successful reflected XSS attack can potentially:

- Execute arbitrary JavaScript in the victim's browser
- Modify page content
- Perform actions using the victim's privileges
- Access data available to JavaScript
- Facilitate phishing or session-related attacks

Remediation

Developers should:

- Apply context-specific output encoding.
- Properly encode quotation marks when inserting untrusted data into attributes.
- Avoid placing untrusted data directly into executable HTML contexts.
- Validate user input where appropriate.
- Use security headers such as CSP as defense in depth.

OWASP Reference

OWASP Top 10: A03 – Injection

Related vulnerability: Cross-Site Scripting (XSS)

Conclusion

This lab demonstrates why HTML-encoding angle brackets alone is not sufficient to prevent XSS. Security controls must account for the exact output context. Attribute values require appropriate attribute-context encoding, including safe handling of quotation marks.

GitHub Commit Message

Add reflected XSS attribute report